

INDUSTRIAL SURFACE DEFECT INSPECTION SYSTEM

Phase 3 — Implementation, Testing and Dataset Report

Team Name: VISION 404
Team Leader: Kanishk Rungta
Team Member: Nethi Kushala Kumar

1. Introduction

Phase 1 set out the requirements. Phase 2 set out the detailed design, the interface and the acceptance test plan. Phase 3 is the build: the application described in Phase 2 now exists, runs, and is covered by an automated test suite. This report is short on purpose. The deliverable is the code, and this document is a map of it — what was built, how it is structured, what is tested, what data it was tested against, and what remains open.

The application is a local web interface for the inspection pipeline. It runs today against a deterministic mock provider with no ONNX file present, and switches to the real pipeline through one configuration change. That split was deliberate: it let the interface, the database, the exports and the tests be finished and reviewed while the final weights were still being trained.

1.1 What is delivered

Item	Delivered
Application	FastAPI + Jinja2 + SQLite, 60 Python modules, six screens, 21 API routes
Database	Eight tables, 11 indexes, foreign keys enforced, append-only
Test suite	250 automated tests, 89% statement coverage, lint clean, about 110 s to run
Dataset	Deterministic generated corpus: 85 inspections, 101 regions, 122 images, 3 batch runs
Documentation	Five documents in the repository, plus 20 screenshots of every screen and state
Deployment	Runs locally; The station target is unchanged

2. Changes Since Phase 2

Building the application surfaced a small number of decisions that differ from the Phase 2 design. They are recorded here rather than left for a reader to discover.

What changed	Why
The Live screen orders by inspection ID, not capture time.	“The item currently at the station” means the most recently processed part. Ordering by capture time would let a re-processed older image appear to be the live one. History still orders by capture time, which is the right axis there.

What changed	Why
Stored image paths are relative to the data root, not absolute.	An absolute path pins the database to the machine that wrote it. A database restored into a different folder, or read after a deployment, would lose every image.
A serverless deployment mode was added.	Only so the interface can be shown to reviewers from a URL. It is a demo mode, not a station mode, and Section 9 states plainly what it cannot do.
app/postprocess.py is a minimal stub in this repository.	The inference core was not part of the Phase 3 handover, so the application had nothing to import thresholds from. The stub holds the Profile values and nothing else — no softmax, no class rule, no geometry — precisely so it cannot become a second implementation. The pipeline's own module replaces it wholesale.
The canonical record key names are unverified.	ARCHITECTURE.md §8.1 defines the record and was not supplied. The real provider maps it through a documented alias table and fails loudly rather than inventing values, but the aliases must be checked against one real Inspector output before the first real run. This is the highest-risk open item.

3. Coding Standards and Structure

3.1 Layering

Four layers, each talking only to the one below, exactly as Phase 2 specified. Routes render and serialise. Services hold the rules. Repositories hold the SQL. Providers are the boundary to the inference pipeline. No layer reaches past the next one down: a route never writes SQL, a template never decides what a status means, and nothing above the provider boundary imports onnxruntime.

Package	Responsibility	Lines
app/routes	HTTP: six page routes, 21 API routes, media serving	750
app/services	Inspection, batch, history, material, status, export logic	1,292
app/repositories	All SQL. One shared filter clause for history, exports and cards	544
app/providers	Provider protocol, mock provider, real adapter, asset generation	1,207
app/database	Schema, connection, migrations, deterministic seeder	411
app/schemas	Typed request and response models, enforced on the API boundary	239
app/templates, app/static	Six screens and partials; plain CSS and plain JavaScript	1,795
tests	250 automated tests	2,159

3.2 Standards applied

Standard	How it is applied
Linting and style	ruff with E, F, W, I, UP, B, C4 and SIM rule sets; the tree is clean. Configuration is in pyproject.toml, so the same rules run locally and in CI.

Standard	How it is applied
Type annotations	Every function signature is annotated. Pydantic models type the configuration and the API boundary; dataclasses type the provider results.
Single source of truth	Thresholds exist in exactly one module. A test fails the build if a threshold literal appears in any template or script.
Configuration, not constants	Paths, station ID, provider, seed and poll interval come from the environment. Nothing operational is hardcoded.
Plain SQL, no ORM	A two-person team maintains this. Plain SQL means less to learn and nothing hidden behind a query builder.
Docstrings that give reasons	Module and function docstrings state why a rule exists, not what the next line does — for example why the class decision is not argmax.
No dead code	The typed response models are enforced on live routes rather than kept as documentation; coverage is used to find modules that nothing exercises.

3.3 Agile increments

The build ran as vertical slices. Each increment ended with the application running and its tests passing, so there was never a state where a screen existed but its data did not.

Increment	Outcome
1. Data foundation	Schema, connection, repositories, deterministic seeder. Verified by seeding a full database before any screen existed.
2. Provider boundary	Protocol, mock provider, real adapter. Fixed the integration contract before anything depended on it.
3. Services and API	Inspection, history, batch, material, status, export. Exercised through the API before any HTML was written.
4. Screens	Six screens against the Phase 2 wireframes, then the responsive and accessibility pass.
5. Test and hardening	Security, content rules, browser tests; then the findings they produced were fixed.
6. Packaging	Documentation, screenshots, deployment build.

4. User Interface Implementation

All six screens from Phase 2 are implemented and match the wireframes. The interface is read-only: it reports what was found and shows the evidence.

Screen	Route	What it does
Live Inspection	/live	Overlay at source resolution, result banner, region evidence table. Refreshes every 2.5 s.
Region Detail	/regions	Region list with previous and next, cropped and enlarged region, measurements and reproducibility panel.

Screen	Route	What it does
Batch Run and Report	/batch	Folder run with progress, five summary cards, results table, CSV and JSON export, status filters.
Inspection History	/history	All six filters on one row, pagination, filtered export, row opens the original evidence.
Materials and Thresholds	/materials	Measured coverage per material, active thresholds read-only, model in use.
System Status	/status	Seven checks with state, detail, recommended action and check time; fault simulation in demo mode.

4.1 What the interface refuses to show

Three rules from Phase 2 shape more of the design than any styling decision, and each is now enforced by a test rather than by discipline.

Rule	How it is enforced
No confidence percentage	The scores are uncalibrated. No API response carries a score field, and 38 content tests search every rendered page for confidence, probability and percentage wording.
No accept or reject verdict	The database CHECK constraint permits only four statuses, so a verdict cannot even be stored. A test scans every rendered page for verdict wording.
Clean is not “could not process”	Separate statuses, separate wording, separate banner treatment, separate export rows. A file that could not be read is never counted as clean.

The class label is marked provisional everywhere it appears, because typing is about 49% correct on crack pixels. The mask is the primary output and the interface says so.

Surface Defect Inspection

line-1-cam-A

Live

Regions

Batch

History

Materials

Status

Live Inspection — line-1-cam-A

DEMO MODE

Next inspection

☐ Advance automatically

Camera: OK · Model: v8a · 5.8 MB · hash 9f2c... · Profile: all materials v2 · Database: OK · Disk: 4% used · Last latency: 29.0 ms

DEGRADED Degraded: Model file / hash, Inference provider. [Open Status](#)

3 REGIONS FOUND

1 crack, 2 scratch

Regions

#	Class	Length	Width	Area	
1	crack	313 px	8 px	2,531	Open
2	scratch	183 px	4 px	739	Open
3	scratch	173 px	11 px	1,911	Open

Class label is provisional — see Materials for measured coverage by material. [Materials](#)

Open region detail

View history

1 crack

2 scratch

3 scratch

Product batch-82/item-23 · 06 Aug 02:11:45 · steel · line-1-cam-A

Overlay drawn at source resolution, not 256×256.

Figure 1. Live inspection — overlay at source resolution, region evidence, no score column.

Surface Defect Inspection

Line-1-cam-A

Live

Regions

Batch

History

Materials

Status

Inspection History

Camera: OK

Model: v8a

5.8 MB

hash 9f2c...

Profile: all materials v2

Database: OK

Disk: 4% used

Last latency: 29.0 ms

DEGRADED

Degraded: Model file / hash, Inference provider: [Open Status](#)

Filters

Date from

mm/dd/yyyy

Date to

mm/dd/yyyy

Material

All

Status

All

Class

All

Station

All

Product ID

batch-77*

Apply

Clear

143 matching inspections

Timestamp	Product ID	Material	Status	Regions	Max length	Model	Station
06 Aug 02:11:45	batch-82/item-23	steel	regions found	3	313 px	v8a	line-1-cam-A
06 Aug 02:10:22	2026-08-10/IMG_3124	plastic	failed to read	—	—	v8a	line-1-cam-A
06 Aug 02:10:22	2026-08-10/IMG_3123	plastic	failed to read	—	—	v8a	line-1-cam-A
06 Aug 02:10:22	2026-08-10/IMG_3122	plastic	failed to read	—	—	v8a	line-1-cam-A
06 Aug 02:10:22	2026-08-10/IMG_3121	plastic	regions found	3	230 px	v8a	line-1-cam-A
06 Aug 02:10:20	2026-08-10/IMG_3120	plastic	clean	0	—	v8a	line-1-cam-A
06 Aug 02:10:19	2026-08-10/IMG_3119	plastic	regions found	2	240 px	v8a	line-1-cam-A
06 Aug 02:10:18	2026-08-10/IMG_3118	plastic	regions found	2	109 px	v8a	line-1-cam-A
06 Aug 02:10:16	2026-08-10/IMG_3117	plastic	regions found	3	221 px	v8a	line-1-cam-A
06 Aug 02:10:16	2026-08-10/IMG_3116	plastic	regions found	3	247 px	v8a	line-1-cam-A
06 Aug 02:10:15	2026-08-10/IMG_3115	plastic	regions found	3	439 px	v8a	line-1-cam-A
06 Aug 02:10:14	2026-08-10/IMG_3114	plastic	regions found	4	300 px	v8a	line-1-cam-A
06 Aug 02:10:12	2026-08-10/IMG_3113	plastic	regions found	2	255 px	v8a	line-1-cam-A
06 Aug 02:10:12	2026-08-10/IMG_3112	plastic	regions found	2	290 px	v8a	line-1-cam-A
06 Aug 02:10:11	2026-08-10/IMG_3111	plastic	processing failed	—	—	v8a	line-1-cam-A
06 Aug 02:10:09	2026-08-10/IMG_3110	plastic	regions found	2	206 px	v8a	line-1-cam-A
06 Aug 02:10:09	2026-08-11/IMG_3126	ceramic	failed to read	—	—	v8a	line-1-cam-A
06 Aug 02:10:09	2026-08-11/IMG_3125	ceramic	failed to read	—	—	v8a	line-1-cam-A
06 Aug 02:10:09	2026-08-11/IMG_3124	ceramic	failed to read	—	—	v8a	line-1-cam-A
06 Aug 02:10:09	2026-08-11/IMG_3123	ceramic	regions found	1	297 px	v8a	line-1-cam-A
06 Aug 02:10:08	2026-08-11/IMG_3122	ceramic	clean	0	—	v8a	line-1-cam-A
06 Aug 02:10:07	2026-08-11/IMG_3121	ceramic	regions found	3	230 px	v8a	line-1-cam-A
06 Aug 02:10:06	2026-08-11/IMG_3120	ceramic	clean	0	—	v8a	line-1-cam-A
06 Aug 02:10:06	2026-08-11/IMG_3119	ceramic	regions found	2	240 px	v8a	line-1-cam-A
06 Aug 02:10:05	2026-08-11/IMG_3118	ceramic	regions found	2	109 px	v8a	line-1-cam-A

Showing 1–25 of 143

Previous

1

2

3

...

6

Next

Export CSV

Export JSON

The export uses the same filter as the table above.

Figure 3. Inspection history — all six filters on one row, filters carried in the query string.

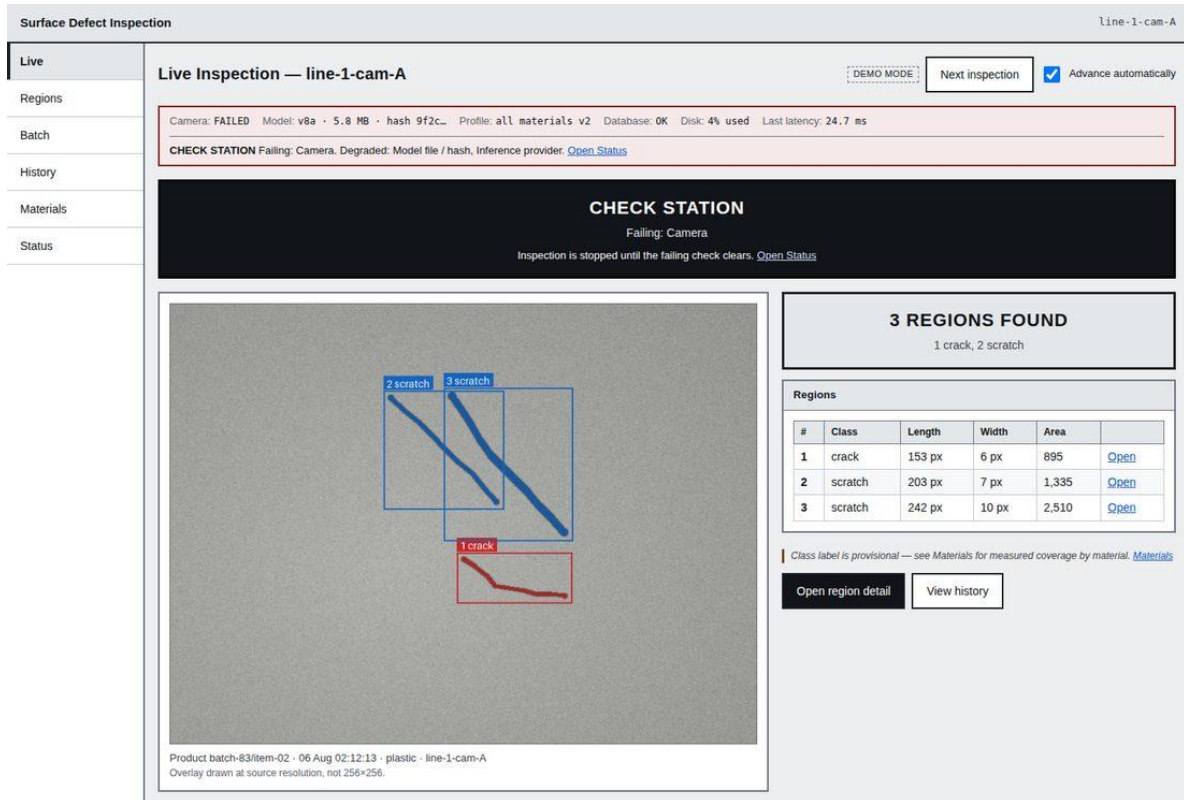


Figure 4. A failing station check replaces the result with CHECK STATION and stops inspection.

5. Scalable and Sustainable Design

5.1 Database

Property	Design
Indexes	Eleven, covering every history filter, the batch report join and the region joins. Filtering is index-supported rather than a table scan.
Append-only	Nothing is updated or deleted. Re-running an image writes a new row; re-tuning thresholds writes a new profile version, so an old inspection still resolves to the thresholds that produced it.
Growth	About 1 KB per inspection including regions — roughly 30 MB a day at one part per second. Older months detach into archive files.
Denormalisation, once	inspection.region_count is stored so listing thousands of rows does not count regions on every query. A test asserts it always agrees with the region table.
Portability	Image paths are relative to the data root, so a database copied to another machine still finds its images.
Extension	A third defect class is one row in defect_class and a retrained head. A verdict, if tolerances ever arrive, is one module and one column; nothing downstream assumes a verdict exists.

5.2 Code

Property	Design
Provider boundary	Swapping mock for real changes one environment variable. No service, route, template or stylesheet changes.
One filter clause	History, the summary cards and both exports are answered by the same SQL clause, so totals cannot drift apart as filters are added.
Stateless services	No module-level mutable state except the batch progress registry, so more worker processes on the same machine is a deployment change, not a rewrite.
Horizontal scale	One unit per station, sharing nothing at runtime. A reporting machine copies each unit's database on a schedule; the line never waits for it.

5.3 Tests

Tests are written to survive the data changing. They compare against independently computed sets rather than fixed row counts, seed their own temporary database rather than the working one, and are parametrised so a new material or a new status extends a table instead of adding a test.

6. Test Implementation and Automation

250 automated tests, **89% statement coverage** of the application package, running in about 110 seconds. The suite needs no camera, no GPU, no model file and no network.

Area	Tests	What it proves
Content rules	38	No confidence wording, no verdict wording, no threshold literal in any template or script, limitations visible, accessibility basics
Page routes	33	Every screen renders, navigation marks one current item, the health strip is on every screen, no external origin
Browser (Playwright)	23	Live updates, region navigation, filtering, batch cards, responsive layout at five widths
Mock provider	22	Determinism, all six states, region floors, real decode failures, source-resolution overlays
Batch service	18	Totals reconcile, failures never counted as clean, dry run, path safety
Status and model hash	18	Seven checks, fault simulation, hash mismatch stops inspection
History	17	Every filter alone and combined against independently computed sets, pagination covers each row once
Security	16	Path traversal, invalid files, decode limits, demo controls refused in real mode
API	15	Live payload, demo advance, listing, detail, region detail, structured errors
Database	13	Schema, foreign keys enforced, constraints, indexes, seed integrity
Deployment	13	Portable paths, read-only filesystem behaviour, deployment configuration

Area	Tests	What it proves
Materials	12	Coverage read from the metrics file, thresholds follow the module when re-tuned
Exports	12	CSV and JSON totals recalculate to the displayed totals

6.1 Acceptance case status

Phase 2 listed 25 acceptance cases. Those that test the application layer are automated now. Those that measure the model cannot be run until the trained weights and the inference core are in the repository, and are listed as blocked rather than quietly dropped.

Status	Cases
Automated and passing	T-06 thresholds not duplicated; T-08 clean versus unreadable; T-14 stored record fields; T-16 batch export totals; T-17 history filters; T-21 no score or verdict on any page; T-22 coverage from the metrics file; T-23 model hash mismatch stops inspection
Partly automated	T-13 overlay at source resolution (asserted for the mock path); T-20 offline operation (no external origin in any page; the system-call trace is not automated)
Blocked on the model	T-01 to T-05, T-07, T-09 to T-12, T-15, T-18, T-19, T-24 — all measure the pipeline or the dataset gate
Manual by design	T-25, the physical rig walkthrough with reference targets and five users

Phase 3 also added test areas Phase 2 did not list, because building the application created them: path traversal and media access, responsive layout at five widths, forbidden-wording scanning of rendered pages, and database portability across machines.

6.2 Automation

Stage	Command
Lint and import order	ruff check .
Full suite	pytest
Without a browser	pytest -m "not ui"
Coverage	pytest --cov=app --cov-report=term
Screenshots of every screen	python -m scripts.capture_screenshots

The stages run in order and a failure blocks the merge. Browser tests skip automatically where no browser binary is installed, so the suite still runs on a machine with only the Python dependencies. When a bug is found, a test that fails before the fix and passes after it is added and kept.

7. Dataset

The application requires a corpus that exercises every state handled by the user interface, database, exported reports, and batch-processing pipeline. Because proprietary factory images

cannot be distributed with the application, a **fully deterministic synthetic validation corpus** is generated instead.

Background surfaces are procedurally synthesised from a seeded generator to resemble common industrial materials, including brushed steel with directional machining marks, speckled ceramic, matte plastic, epoxy resin, glass and non-steel metallic finishes. Illumination varies spatially to emulate realistic inspection stations where lighting is never perfectly uniform.

Defects are generated procedurally rather than pasted from photographs. **Cracks** are represented as irregular branching polylines with variable width and tortuosity, while **scratches** are thinner, straighter and exhibit lower curvature. This provides realistic geometric variation while remaining completely reproducible from the generation seed.

Validation Corpus Coverage

Dimension	Coverage
Volume	85 inspections, 101 defect regions, 122 generated images, 25 batch summary files
Inspection Statuses	All schema-defined states represented: 44 inspections with detected defects, 22 clean inspections, 13 acquisition failures, and 6 processing failures
Defect Classes	55 crack regions and 46 scratch regions , including inspections containing both defect types
Material Coverage	All six supported materials represented: Steel (32), Ceramic (17), Plastic (16), Epoxy (8), Glass (7) and Non-steel Metal (5)
Operational Coverage	Data spans 3 inspection stations, 9 operating days, and 3 batch processing runs
Failure Modes	8 distinct pipeline failures: zero-byte file, truncated image, unsupported media type, decode failure, camera timeout, inference failure, post-processing failure and overlay write failure
Resolution Coverage	4 input resolutions , ranging from 640 × 480 to 960 × 720 , verifying resolution-independent overlay generation

7.1 Deliberate Edge Cases

Each generated batch intentionally contains malformed inputs to exercise the application's error-handling paths under realistic conditions. Three unreadable files are inserted into every batch:

- a zero-byte image,
- a deliberately truncated PNG,
- and a plain-text file renamed with a `.png` extension.

These files trigger genuine decoder failures rather than simulated error flags, ensuring the complete acquisition and decoding pipeline is exercised.

Very small generated defects are filtered using the same minimum-area and skeleton-length thresholds employed by the inspection pipeline. Consequently, changing the application's filtering parameters automatically changes the generated validation corpus as well, keeping both systems consistent.

To guarantee complete functional coverage, the generator forces at least one example of every inspection state into the beginning of each generated dataset. Even small generated corpora therefore exercise every application state.

7.2 Reproducibility

The entire validation corpus is deterministic. Inspection status, generated geometry, metadata and image pixels are all pure functions of the configured random seed. Using the same seed always regenerates an identical dataset on any machine without requiring network access.

Inspection timestamps are generated relative to a configurable anchor date (defaulting to the current date) so that generated histories appear realistic while remaining reproducible when the anchor date is fixed.

Determinism is verified through automated tests rather than assumed by design.

7.3 Scope of the Validation Dataset

This corpus exists solely to validate the behaviour of the application.

It is **not** used for model training, model validation or performance evaluation, and no accuracy, precision or segmentation metric reported by the project is derived from it. Those results are obtained exclusively from the frozen real-image evaluation datasets described in Phase 2.

Accordingly, the application clearly identifies these inspection results as **generated demonstration data** intended to exercise the user interface, database, reporting pipeline and failure-handling logic rather than to represent measured model performance.

8. Documentation

Document	Purpose
UI_README.md	Installation, configuration, application startup, database seeding, testing, and developer workflow.
ARCHITECTURE.md	Software architecture, system workflow, component interactions, module responsibilities, and scalability considerations.
DATABASE.md	Database schema, relationships, indexing strategy, normalisation, persistence, and backup considerations.
MODEL_INTEGRATION.md	Integration interface between the application and the ONNX inference engine, provider abstraction, model deployment, and replacement procedure.

DATASET.md	Dataset governance, provenance, versioning, quality assurance, reproducibility, and data lifecycle documentation supporting application validation.
DEPLOYMENT.md	Deployment procedures for both the industrial inspection station and the demonstration environment, including operational limitations.
TEST_PLAN.md	Acceptance test cases, verification procedures, expected outcomes, and requirements traceability.
REQUIREMENTS.md	Functional and non-functional requirements with implementation status and traceability to project deliverables.
DECISIONS.md	Architecture Decision Records (ADRs) documenting engineering decisions, design rationale, and implementation trade-offs.
.env.example	Documents every configurable environment variable, default value, and deployment parameter required for reproducible installation and configuration. Configuration is externalised from source code to simplify deployment across environments.
docs/screenshots/	A complete visual record of the application, including desktop and mobile layouts, normal operation, filtered views, empty states, clean inspections, processing failures, and system fault scenarios. These provide implementation evidence, regression references, and user documentation.

Documentation is organised to maximise maintainability, reproducibility, and traceability. Architectural decisions, system design, database structure, deployment procedures, model integration, dataset governance, testing, and requirements are documented independently so that each subsystem can be understood and maintained without relying solely on source code.

9. Other Observations

9.1 Deployment

The application runs locally with two commands and deploys to a serverless host for demonstration. That demo mode required three changes — writes redirected to the one writable path, a database seeded at build time because a cold start has seconds and seeding takes minutes, and batch work moved onto the request thread because a background thread is frozen once a response is sent. None of them touched the screens, the schema or the provider boundary.

A serverless deployment cannot be the station deployment, and this is not a limitation to work around. The inference path deliberately opens no socket because factory images must not leave the site; that requirement and a public host are incompatible by design. The demo shows the interface. The station stays on the CPU-only industrial PC, offline, with a database that persists.

9.2 What building it taught us

Two things. First, the rules that are easiest to state are the easiest to lose: “do not show a confidence percentage” survives one review and then quietly reappears in a helpful tooltip. Encoding each such rule as a test that scans the rendered output turned them from intentions into properties of the build. Second, mock mode was worth more than expected. Because every state — including failures that are hard to reproduce on demand — could be produced deterministically, the failure paths got the same attention as the happy path instead of being written last and tested least.

9.3 Open items

Item	Position
Record key names unverified	The highest-risk item. Run one real image through Inspector, print the record and confirm it against the alias table before the first real run.
postprocess.py is a stub	Replace it with the pipeline's module and confirm the four threshold values match.
Model-side acceptance cases blocked	Fourteen cases measure the pipeline and cannot run until the weights and the inference core are present.
Retention not automated	The 48-hour and 90-day image retention is specified and documented but is not yet a scheduled job.
Class typing remains the open defect	Around 96% detection, about 49% correct typing on crack pixels. More masks on non-steel materials is the only real fix and is the top data priority.
No accounts, so no attribution	The system cannot tell you who did what. A deliberate consequence of dropping the verdict, and a real limitation if a customer needs it.